Heavens Light is Our Guide

Rajshahi University of Engineering and Technology

Department of Electrical & Computer Engineering

Course Title

Data Structure & Algorithms Sessional

Course No: ECE 2104

Lab Report -04

Name of the Experiment: Queue Implementation using Linked List (enqueue(), dequeue(), front(), display())

Date of Submission: 25 July 2026

Submitted By:

Uddipon Chowdhury
Roll: 2410019
Registration No: 1072

Submitted To:

Md. Faysal Ahamed
Assistant Professor
ECE, RUET

Theory

1.1 Data Structure and Linear Data Structure

A data structure is a systematic scheme for organizing, storing, and managing data in a computer so that it may be accessed and modified efficiently [1]. The choice of data structure directly influences the efficiency of the algorithms that operate on it, since operations such as insertion, deletion, searching, and traversal are only as fast as the underlying organization permits. Data structures are broadly categorized into linear and non-linear types. In a linear data structure, data elements are arranged in a sequential order such that every element, except the first and the last, is connected to exactly one predecessor and one successor, and the structure can be traversed in a single logical run [2]. Arrays, linked lists, stacks, and queues are the principal examples of linear data structures, and the queue — the subject of this report — is one of the most widely used among them.

1.2 Abstract Data Type (ADT)

An Abstract Data Type (ADT) is a mathematical model of a data structure that specifies the set of values it can hold and the operations that can be performed on it, without dictating how those operations are actually realized in code [1]. The ADT view separates interface from implementation: a user of a queue ADT only needs to know that it supports insertion at the rear and removal at the front, and need not be concerned with whether the underlying storage is an array, a linked list, or some other structure. This separation of concerns is central to modular software design and is the reason the queue can be implemented equally well using arrays or linked lists while presenting an identical external behaviour.

1.3 Queue and the FIFO Principle

A queue is a linear ADT in which elements are inserted at one end, called the rear (or tail), and removed from the other end, called the front (or head) [3]. This restriction on where insertion and deletion may occur enforces the First-In-First-Out (FIFO) discipline: the element that has been in the queue the longest is always the first to be removed. The FIFO principle mirrors many real-world waiting-line phenomena, which is why the queue is the natural abstraction for modelling processes that must be served strictly in the order they arrive. The fundamental operations defined by the queue ADT are enqueue (insertion at the rear), dequeue (removal from the front), front/peek (inspection of the front element without removing it), isEmpty (a predicate reporting whether the queue holds any element), and display (a non-standard but pedagogically useful operation that traverses and prints the current contents) [4].

1.4 Linked List, Node Structure, and Dynamic Memory Allocation

A linked list is a linear data structure composed of a sequence of nodes, where each node stores a data field and a pointer to the next node in the sequence [2]. Unlike an array, a linked list does not require its elements to occupy contiguous memory locations; instead, logical ordering is maintained entirely through pointers. A node is typically represented as a small class or structure containing a data member of the type being stored and a pointer member, conventionally named next, that references the following node or is set to nullptr when no successor exists.

Nodes are created at run time using dynamic memory allocation, performed in C++ with the new operator, and are released with delete when they are no longer required [5]. Because memory is requested from the heap only as needed, a linked list can grow or shrink during program execution without any pre-declared

upper bound, in direct contrast to a fixed-capacity array. This property is the principal motivation for implementing the queue ADT over a linked list rather than an array: the resulting structure is not constrained by a fixed capacity and therefore does not suffer from the artificial "queue full" condition that afflicts a naive array-based circular queue.

1.5 Queue Implementation Using a Linked List

When a queue is implemented using a singly linked list, two external pointers are maintained: front, which references the node at the head of the list (the next element to be served), and rear, which references the node at the tail of the list (the most recently inserted element) [4]. Maintaining both pointers is essential for efficiency — without a dedicated rear pointer, enqueue would require traversing the entire list to locate the last node before a new node could be appended, degrading the operation from constant to linear time.

The four operations required in this report are implemented as follows over this structure:

- Enqueue: a new node is allocated and linked after the current rear node; the rear pointer is then advanced to the new node. If the queue was empty prior to insertion, both front and rear are set to point to the newly created node.
- Dequeue: the node referenced by front is logically removed by advancing front to front->next, after which the original front node is deallocated. If the queue becomes empty as a result (front becomes nullptr), rear is also reset to nullptr to keep the two pointers consistent.
- Front: the data field of the node referenced by front is returned without altering the structure of the queue in any way.
- Display: the queue is traversed from front to rear using a temporary pointer, printing the data field of each node in order, which by construction is also the order in which elements will be served.

Because each of these operations acts only on the ends of the list — never requiring an internal traversal — a linked-list-based queue realizes every core operation in constant time, independent of how many elements the queue currently holds.

1.6 Queue Overflow and Underflow

Two exceptional conditions are traditionally associated with queue operations. Overflow refers to an attempt to enqueue an element when the queue has no remaining capacity; in a linked-list implementation this condition is effectively eliminated for all practical purposes, since a new node is requested from the heap only at the moment it is needed, and failure can occur only if the heap itself is exhausted — an operating-system-level condition rather than a structural limitation of the queue [5]. Underflow, by contrast, refers to an attempt to dequeue or inspect the front of an empty queue, and remains a meaningful condition regardless of the underlying storage scheme. A robust implementation must therefore detect the empty state (front == nullptr) before performing a dequeue or front operation and respond by raising an exception rather than dereferencing a null pointer, which would otherwise invoke undefined behaviour [1]. This report uses C++ exception handling — a try/catch block around a thrown std::underflow_error — to satisfy this requirement in a manner consistent with modern C++ practice.

1.7 Time and Space Complexity

For a linked-list-based queue holding n elements, the asymptotic costs of the four operations considered in this report are summarized below.

Operation	Time Complexity	Auxiliary Space
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Front	O(1)	O(1)
Display	O(n)	O(1)

Enqueue, dequeue, and front each touch only a single pointer (rear or front) and therefore execute in constant time and constant auxiliary space, since no additional storage proportional to n is allocated during the operation [2]. Display must visit every node exactly once to print its contents and is consequently linear in time, although it too requires only a single temporary pointer and hence constant auxiliary space. It is worth noting that the linked-list realization achieves O(1) enqueue and dequeue without the modular ("circular") index arithmetic that an array-based queue requires to avoid wasting the slots freed by earlier dequeues [3].

1.8 Advantages, Limitations, and Applications

The linked-list-based queue offers several advantages over its array-based counterpart. Its size grows and shrinks dynamically with actual usage, so memory is never reserved speculatively and no fixed capacity need be chosen in advance [5]. Enqueue and dequeue remain O(1) regardless of how many elements have been processed over the queue's lifetime, and the structure never exhibits the "false full" condition that a naive (non-circular) array queue suffers from once the rear index reaches the end of the array even while free slots exist at the front.

Its principal limitations are the extra memory consumed by the next pointer in every node relative to a packed array, the absence of random access to arbitrary elements, and a marginally higher constant-factor cost per operation due to dynamic memory allocation and pointer indirection [1].

Queues implemented in this manner are used extensively in computing systems: CPU and disk scheduling algorithms that must service requests in arrival order, buffering of data streams between producers and consumers operating at different rates, breadth-first traversal of graphs and trees, print spoolers, and the handling of asynchronous events and messages in operating systems and networking stacks all rely on the FIFO ordering that the queue ADT guarantees [3][4].

Task 01 — Enqueue (Push)

Problem Definition

Implement the **enqueue (push)** operation of a queue using a singly linked list. The program should insert new elements at the **rear** of the queue while maintaining the **FIFO (First-In-First-Out)** property. If the queue is initially empty, both the **front** and **rear** pointers should reference the newly inserted node. The program should display the updated queue after each insertion.

Algorithm

1. Start.
2. Create a new node and store the element to be inserted.
3. Set the next pointer of the new node to `nullptr`.
4. Check whether the queue is empty.
5. If the queue is empty, set both `front` and `rear` to the new node.
6. Otherwise, attach the new node after the current `rear` node.
7. Update the `rear` pointer to the newly inserted node.
8. Increment the queue size.
9. Display the updated queue.
10. Stop.

Complexity Analysis

Complexity	Value
Time Complexity	O(1)
Space Complexity	O(1)

Code

```
#include <iostream>
using namespace std;

template <typename T>
class Node
{
public:
    T data;
    Node<T>* next;

    Node(T value)
    {
```

```
        data = value;
        next = nullptr;
    }
};

template <typename T>
class Queue
{
private:
    Node<T>* front;
    Node<T>* rear;
    int count;

public:
    Queue()
    {
        front = nullptr;
        rear = nullptr;
        count = 0;
    }

    void enqueue(T value)
    {
        Node<T>* newNode = new Node<T>(value);

        if (rear == nullptr)
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }

        count++;
    }

    void display()
```

```
    {
        if (front == nullptr)
        {
            cout << "Queue is Empty.\n";
            return;
        }

        Node<T>* current = front;

        cout << "Front -> ";

        while (current != nullptr)
        {
            cout << current->data;

            if (current->next != nullptr)
                cout << " -> ";

            current = current->next;
        }

        cout << " <- Rear\n";
    }

    int size()
    {
        return count;
    }
};

int main()
{
    Queue<int> q;

    int values[] = {15, 27, 8, 42, 19};

    cout << "========== ENQUEUE OPERATION ==========\n\n";

    for (int value : values)
```

```
    {
        cout << "Inserting " << value << " into the queue...\n";

        q.enqueue(value);

        q.display();

        cout << "Current Queue Size : "
             << q.size() << "\n";

        cout << "----------------------------------------\n";
    }

    return 0;
}
```

Program Output

```
========== ENQUEUE OPERATION ==========

Inserting 15 into the queue...
Front -> 15 <- Rear
Current Queue Size : 1
----------------------------------------
Inserting 27 into the queue...
Front -> 15 -> 27 <- Rear
Current Queue Size : 2
----------------------------------------
Inserting 8 into the queue...
Front -> 15 -> 27 -> 8 <- Rear
Current Queue Size : 3
----------------------------------------
Inserting 42 into the queue...
Front -> 15 -> 27 -> 8 -> 42 <- Rear
Current Queue Size : 4
----------------------------------------
Inserting 19 into the queue...
Front -> 15 -> 27 -> 8 -> 42 -> 19 <- Rear
Current Queue Size : 5
----------------------------------------
```

Discussion

The **enqueue** operation inserts a new element at the **rear** of the queue while preserving the **First-In-First-Out (FIFO)** principle. If the queue is initially empty, both the `front` and `rear` pointers are initialized to the new node. Otherwise, the new node is linked after the current rear node, and the rear pointer is updated accordingly. Since the operation modifies only the rear end of the linked list without traversing existing nodes, it executes in **constant time, O(1)**. The sample output confirms that each inserted element is appended to the rear of the queue while maintaining the correct order of elements.

Task 02 — Dequeue (Pop)

Problem Definition

Implement the **dequeue (pop)** operation of a queue using a singly linked list. The program should remove and return the element located at the front of the queue while preserving the **FIFO (First-In-First-Out)** principle. If the queue becomes empty after deletion, both the **front** and **rear** pointers should be updated accordingly. The program should also detect and handle the **queue underflow** condition when a dequeue operation is attempted on an empty queue.

Algorithm

1. Start.
2. Check whether the queue is empty.
3. If the queue is empty, display an underflow message and terminate the operation.
4. Store the address of the front node in a temporary pointer.
5. Store the data of the front node.
6. Move the `front` pointer to the next node.
7. If the queue becomes empty, set the `rear` pointer to `nullptr`.
8. Delete the previous front node.
9. Decrement the queue size.
10. Display the updated queue.
11. Stop.

Complexity Analysis

Complexity	Value
Time Complexity	O(1)
Space Complexity	O(1)

Code

```
#include <iostream>
#include <stdexcept>
using namespace std;

template <typename T>
class Node
{
public:
    T data;
    Node<T>* next;

    Node(T value)
```

```
    {
        data = value;
        next = nullptr;
    }
};

template <typename T>
class Queue
{
private:
    Node<T>* front;
    Node<T>* rear;
    int count;

public:
    Queue()
    {
        front = nullptr;
        rear = nullptr;
        count = 0;
    }

    void enqueue(T value)
    {
        Node<T>* newNode = new Node<T>(value);

        if (rear == nullptr)
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }

        count++;
    }
```

```
    T dequeue()
    {
        if (front == nullptr)
        {
            throw underflow_error("Queue Underflow! Cannot dequeue from an empty
queue.");
        }

        Node<T>* temp = front;
        T value = temp->data;

        front = front->next;

        if (front == nullptr)
            rear = nullptr;

        delete temp;
        count--;

        return value;
    }

    void display()
    {
        if (front == nullptr)
        {
            cout << "Queue is Empty.\n";
            return;
        }

        Node<T>* current = front;

        cout << "Front -> ";

        while (current != nullptr)
        {
            cout << current->data;

            if (current->next != nullptr)
```

```
                cout << " -> ";

            current = current->next;
        }

        cout << " <- Rear\n";
    }

    bool isEmpty()
    {
        return front == nullptr;
    }

    int size()
    {
        return count;
    }
};

int main()
{
    Queue<int> q;

    int values[] = {15, 27, 8, 42, 19};

    for (int value : values)
        q.enqueue(value);

    cout << "========== DEQUEUE OPERATION ==========\n\n";

    cout << "Initial Queue:\n";
    q.display();
    cout << "---------------------------------------\n";

    try
    {
        while (!q.isEmpty())
        {
            cout << "Deleted Element : " << q.dequeue() << endl;
```

```
            q.display();

            cout << "Current Queue Size : "
                 << q.size() << endl;

            cout << "---------------------------------------\n";
        }

        cout << "Attempting one more dequeue...\n";
        q.dequeue();
    }
    catch (underflow_error& e)
    {
        cout << e.what() << endl;
    }

    return 0;
}
```

Program Output

```
========== DEQUEUE OPERATION ==========

Initial Queue:
Front -> 15 -> 27 -> 8 -> 42 -> 19 <- Rear
---------------------------------------
Deleted Element : 15
Front -> 27 -> 8 -> 42 -> 19 <- Rear
Current Queue Size : 4
---------------------------------------
Deleted Element : 27
Front -> 8 -> 42 -> 19 <- Rear
Current Queue Size : 3
---------------------------------------
Deleted Element : 8
Front -> 42 -> 19 <- Rear
Current Queue Size : 2
---------------------------------------
Deleted Element : 42
Front -> 19 <- Rear
Current Queue Size : 1
---------------------------------------
Deleted Element : 19
Queue is Empty.
Current Queue Size : 0
---------------------------------------
Attempting one more dequeue...
Queue Underflow! Cannot dequeue from an empty queue.
```

Discussion

The **dequeue** operation removes the element located at the **front** of the queue, thereby preserving the **First-In-First-Out (FIFO)** principle. After removing the front node, the `front` pointer is updated to the next node. If the queue becomes empty, the `rear` pointer is also reset to `nullptr` to maintain a valid queue structure. Since only the front node is accessed and removed without traversing the linked list, the operation executes in **O(1)** time with **O(1)** auxiliary space. The sample output verifies that elements are removed in the same order in which they were inserted and that the program correctly handles the queue underflow condition by displaying an appropriate error message instead of terminating unexpectedly.

Task 03 — Front

Problem Definition

Implement the **front** operation of a queue using a singly linked list. The program should retrieve and display the element located at the **front** of the queue without removing it. The operation must preserve the **FIFO (First-In-First-Out)** order and correctly handle the **queue underflow** condition when the queue is empty.

Algorithm

1. Start.
2. Check whether the queue is empty.
3. If the queue is empty, display an underflow message and terminate the operation.
4. Access the data stored in the front node.
5. Display the front element without modifying the queue.
6. Stop.

Complexity Analysis

Complexity	Value
Time Complexity	O(1)
Space Complexity	O(1)

Code

```
#include <iostream>
#include <stdexcept>
using namespace std;

template <typename T>
class Node
{
public:
```

```
    T data;
    Node<T>* next;

    Node(T value)
    {
        data = value;
        next = nullptr;
    }
};

template <typename T>
class Queue
{
private:
    Node<T>* front;
    Node<T>* rear;
    int count;

public:
    Queue()
    {
        front = nullptr;
        rear = nullptr;
        count = 0;
    }

    void enqueue(T value)
    {
        Node<T>* newNode = new Node<T>(value);

        if (rear == nullptr)
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }
```

```
        count++;
    }

    T getFront()
    {
        if (front == nullptr)
        {
            throw underflow_error("Queue Underflow! Queue is empty.");
        }

        return front->data;
    }

    T dequeue()
    {
        if (front == nullptr)
        {
            throw underflow_error("Queue Underflow! Queue is empty.");
        }

        Node<T>* temp = front;
        T value = temp->data;

        front = front->next;

        if (front == nullptr)
            rear = nullptr;

        delete temp;
        count--;

        return value;
    }

    void display()
    {
        if (front == nullptr)
        {
```

```
            cout << "Queue is Empty.\n";
            return;
        }

        Node<T>* current = front;

        cout << "Front -> ";

        while (current != nullptr)
        {
            cout << current->data;

            if (current->next != nullptr)
                cout << " -> ";

            current = current->next;
        }

        cout << " <- Rear\n";
    }

    bool isEmpty()
    {
        return front == nullptr;
    }
};

int main()
{
    Queue<int> q;

    int values[] = {15, 27, 8, 42, 19};

    for (int value : values)
        q.enqueue(value);

    cout << "========== FRONT OPERATION ==========\n\n";

    cout << "Current Queue:\n";
```

```
    q.display();

    cout << "\nFront Element : "
         << q.getFront() << endl;

    cout << "\nPerforming one dequeue...\n";

    q.dequeue();

    cout << "\nUpdated Queue:\n";
    q.display();

    cout << "\nFront Element : "
         << q.getFront() << endl;

    while (!q.isEmpty())
        q.dequeue();

    cout << "\nAttempting to access front of an empty queue...\n";

    try
    {
        cout << q.getFront();
    }
    catch (underflow_error& e)
    {
        cout << e.what() << endl;
    }

    return 0;
}
```

Program Output

```
========== FRONT OPERATION ==========

Current Queue:
Front -> 15 -> 27 -> 8 -> 42 -> 19 <- Rear

Front Element : 15

Performing one dequeue...
```

```
Updated Queue:
Front -> 27 -> 8 -> 42 -> 19 <- Rear

Front Element : 27

Attempting to access front of an empty queue...
Queue Underflow! Queue is empty.
```

Discussion

The **front** operation retrieves the element located at the beginning of the queue without removing it, thereby preserving the **FIFO (First-In-First-Out)** property. Unlike the dequeue operation, the queue remains unchanged because only the data stored in the front node is accessed. Since the operation directly references the `front` pointer without traversing the linked list, it executes in **constant time O(1)** with **O(1)** auxiliary space. The output demonstrates that the front element changes only after a dequeue operation and that an appropriate underflow message is displayed when attempting to access the front element of an empty queue instead of causing the program to terminate unexpectedly.

Task 04 — Display

Problem Definition

Implement the **display** operation of a queue using a singly linked list. The program should traverse the queue from **front** to **rear** and display all stored elements in their correct order. If the queue is empty, an appropriate message should be displayed to indicate that no elements are available.

Algorithm

1. Start.
2. Check whether the queue is empty.
3. If the queue is empty, display an appropriate message and terminate the operation.
4. Create a temporary pointer and initialize it to the `front` node.
5. Traverse the linked list until the temporary pointer becomes `nullptr`.
6. Display the data stored in each node while moving the pointer to the next node.
7. Indicate the positions of the `front` and `rear` elements in the output.
8. Display the total number of elements in the queue.
9. Stop.

Complexity Analysis

Complexity	Value
Time Complexity	O(n)
Space Complexity	O(1)

Code

```
#include <iostream>
using namespace std;

template <typename T>
class Node
{
public:
    T data;
    Node<T>* next;

    Node(T value)
    {
        data = value;
        next = nullptr;
    }
};

template <typename T>
class Queue
{
private:
    Node<T>* front;
    Node<T>* rear;
    int count;

public:
    Queue()
    {
        front = nullptr;
        rear = nullptr;
        count = 0;
    }

    void enqueue(T value)
    {
        Node<T>* newNode = new Node<T>(value);

        if (rear == nullptr)
```

```
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }

        count++;
    }

    void display()
    {
        if (front == nullptr)
        {
            cout << "Queue is Empty.\n";
            return;
        }

        Node<T>* current = front;

        cout << "Front -> ";

        while (current != nullptr)
        {
            cout << current->data;

            if (current->next != nullptr)
                cout << " -> ";

            current = current->next;
        }

        cout << " <- Rear\n";
    }

    int size()
    {
```

```
        return count;
    }
};

int main()
{
    Queue<int> q;

    int values[] = {15, 27, 8, 42, 19};

    for (int value : values)
        q.enqueue(value);

    cout << "========== DISPLAY OPERATION ==========\n\n";

    cout << "Queue Contents:\n";
    q.display();

    cout << "\nTotal Elements : "
         << q.size() << endl;

    cout << "\nDisplaying the queue again:\n";
    q.display();

    return 0;
}
```

Program Output

```
========== DISPLAY OPERATION ==========

Queue Contents:
Front -> 15 -> 27 -> 8 -> 42 -> 19 <- Rear

Total Elements : 5

Displaying the queue again:
Front -> 15 -> 27 -> 8 -> 42 -> 19 <- Rear
```

Discussion

The **display** operation traverses the queue from the **front** node to the **rear** node and presents all elements in their correct **FIFO (First-In-First-Out)** order. Unlike enqueue and dequeue operations, the display

operation does not modify the queue; it only reads and prints the stored data. Since every node must be visited once during traversal, the operation has a time complexity of **O(n)**, where *n* is the number of elements in the queue. The output confirms that all elements are displayed sequentially from the front to the rear while preserving the original queue structure.

References

[1] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed. Boston, MA, USA: Pearson, 2014.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[3] E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of Data Structures in C++, 2nd ed. Summit, NJ, USA: Silicon Press, 2007.

[4] R. Lafore, Data Structures and Algorithms in C++, 2nd ed. Indianapolis, IN, USA: Sams Publishing, 2002.

[5] S. Lipschutz, Data Structures (Schaum's Outline Series). New Delhi, India: Tata McGraw-Hill, 2006.